
8

Design with Embedded Processors

- CUT to include only the filter parts

8.1 Filter Design

As an example for an embedded design we use an FIR filter. We will show how steps discussed in Section 8.1 and depicted in **Error! Reference source not found.** can be used to design a processor system that implements our specified digital filter.

In order to show the actual design steps and not overshadow these design steps with use of design tools and utilities, we will use our homemade processor of Chapter 4 (SAYEH) and perform all the design steps manually.

The SAYEH processor we are using for our embedded processor is available in behavioral pre-synthesis Verilog. We will use memory mapped I/O for its interfacing to keep its bussing structure simple. The filter program will be written in C, and we will show a hand translation of it into SAYEH Assembly Language.

8.1.1 Filter Concepts

This section gives a brief overview of filters. We will only discuss filter concepts to the point that we can discuss hardware implementation of digital filters.

8.2.1.1 Analog and Digital Filters. A filter has the property of allowing some frequencies in its incoming waveform, and blocking others. An analog filter uses active or passive components for the job of filtering or eliminating certain frequencies from an incoming waveform. A

simple passive analog filter is an RC circuit (Figure 8.1). This circuit is a low pass filter, meaning that it eliminates high frequency components of its input $f(t)$.

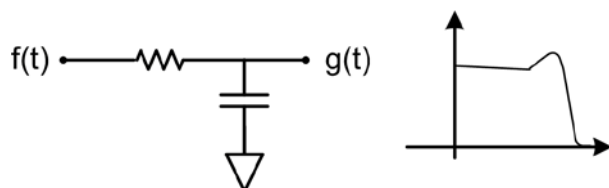


Figure 8.1 Passive Low Pass Filter

Because filters are mainly concerned with frequencies of an input signal, most filter discussions and representations are in the frequency domain. Filter representation in Figure 8.1 indicates that the filter allows lower frequencies to pass through, while higher frequencies are blocked.

An important application of filters is in audio and video processing. Since most such applications are done with digital computers, it is more appropriate to use digital techniques for filtering so that the job of filtering and other audio and video processing can all be done with the same integrated digital system.

8.2.1.2 Sampling. An analog signal to be used with a digital filter must be turned into a series of digital data. This data is obtained by sampling the analog data and then digitizing the sample data as shown in Figure 8.2.

Figure 8.2 shows that an input analog signal is sampled with a sample-and-hold (S&H) circuit. This circuit outputs discrete data. The discrete analog data is then converted to digital data using an Analog-to-Digital converter (A/D). After filtering, and possibly other processings, the processed digital data is turned into analog (e.g., audio or video signals) by a Digital-to-Analog (D/A) converter.

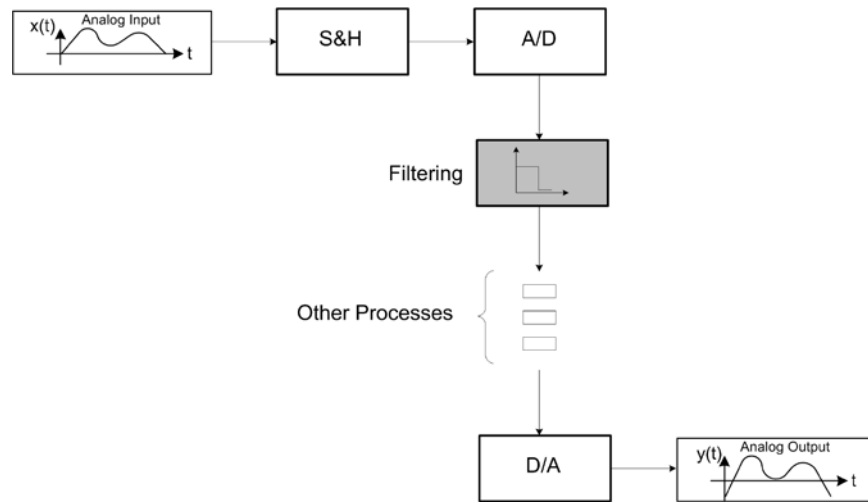


Figure 8.2 Sampling and Digitizing

According to Nyquist, in order to sample a signal without loss of information, the sampling frequency must be faster than twice the highest frequency of the incoming signal. This frequency is referred to as the Nyquist frequency. Signal sampling is done by a train of pulses at a frequency of at least the Nyquist frequency. Figure 8.3 shows the sampling process of an incoming $x(t)$ analog signal.

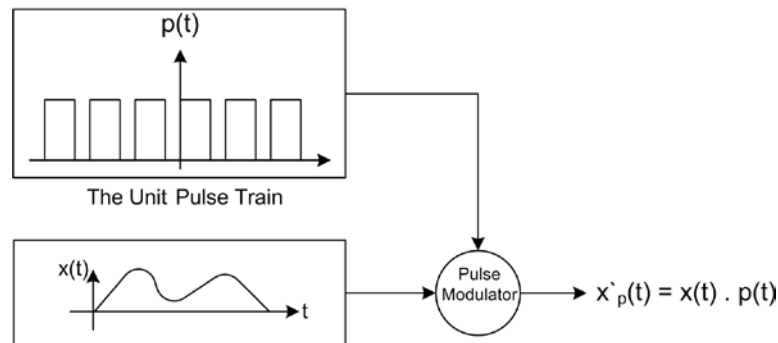


Figure 8.3 Sampling with $P(t)$ Pulse Train

When modulated by a pulse modulator, each sample of an incoming signal becomes a pulse that is fed into the filter. The accumulation of the responses of a filter to the train of pulses that appear on its input becomes the output of the filter. The response of a filter to a unit pulse determines its functionality.

8.2.1.3 Impulse Response. Digital filters are characterized by their response to an impulse. Figure 8.4 shows frequency response of low-pass, high-pass and band pass filters. Also shown here is the impulse response of a low-pass filter. The impulse response is described by a set of coefficients; the number of coefficients determines the order of the filter. The order of a filter determines how accurate filtering is done. This translates to a sharper frequency response for higher order filters.

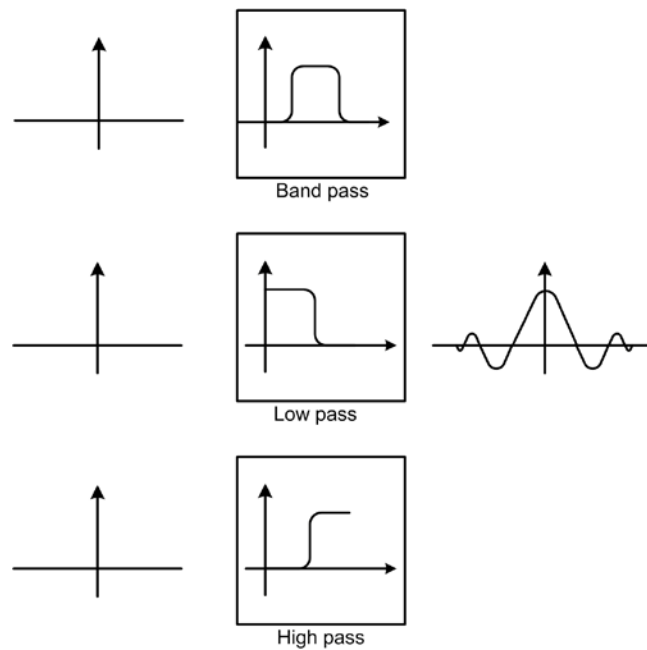


Figure 8.4 Filter Impulse Response

The response of a filter to a series of input samples can be considered as convolution of filter impulse responses to the individual samples. This convolution takes the present and past input samples and output responses into account.

8.2.1.4 Filter Equation. What was described above will be put into an equation form from which the necessary hardware of a filter can be extracted. As discussed, as train of pulses from a sampled input source enter a digital filter, the response of the filter becomes the convolution of previous inputs and previous outputs. Equation below shows a filter output $y[n]$ in terms of its previous inputs, i.e., $x[n-k]$, and its previous outputs, $y[n-j]$. In this equation, input coefficients are indicated by $a[j]$ and output coefficients are $b[k]$.

$$y[n] = \sum a[j] * y[n-j] + \sum b[k] * x[n-k]$$

The block diagram for implementation of this equation is shown in Figure 8.5. This filter has a feedback that causes its impulse response to be infinite in duration. Such a filter is called an infinite impulse response (IIR) filter.

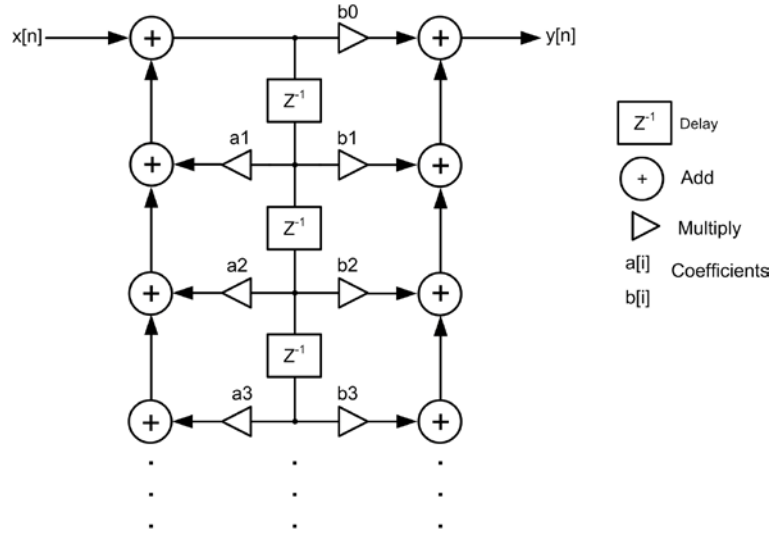


Figure 8.5 An IIR Filter

In Figure 8.5, the delay elements delay input values to be used in the next recursions. This filter is also called a recursive filter.

8.2.1.5 Finite Impulse Response (FIR) Filter. The IIR filter of Figure 8.6 can be simplified by eliminating its feedback. This eliminates output coefficients and simplifies calculation of remaining filter coefficients. Shown below is the resulting equation.

$$y[n] = \sum c[k] * x[n-k]$$

A filter made as such has finite impulse response and is referred to as an FIR or finite impulse response filter. In an FIR filter, filter coefficients can directly be extracted from its impulse response. An FIR filter block diagram is shown in Figure 8.6.

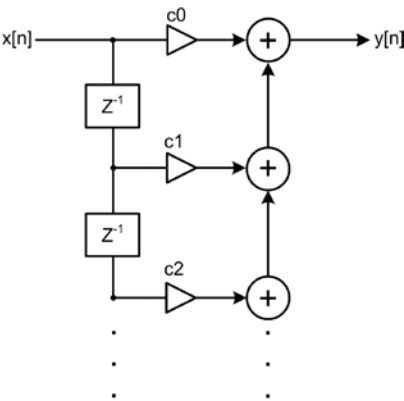


Figure 8.6 An FIR Filter

In the above sections we started with general filter concepts and ended with implementation of an FIR filter. The section that follows discusses alternatives for hardware implementation of this filter.

8.1.2 FIR Filter Hardware Implementation

The discussions of the previous section and filter block diagrams provide a good guideline for designing a digital system for a digital filter. This section shows two possibilities for implementing an FIR filter.

As discussed, filter coefficients are taken from the impulse response of a filter. These coefficients are the main factors in filter design. A generic hardware can be implemented using the structures that we discuss, and it can be configured for a specific filter by changing the coefficients.

The filter we are designing is a 4th order FIR filter with five coefficients that are shown in Figure 8.7. We will show an RTL and a CPU based hardware implementations.

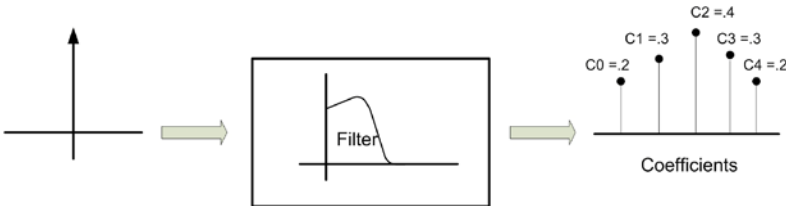


Figure 8.7 Filter to Design

8.2.2.1 FIR Filter RTL Design. Figure 8.9 shows the block diagram of the RT level hardware of a fourth order FIR filter. A series of regis-

ters provide delayed inputs that are multiplied by c_i coefficients and then added together to generate the output. The clock frequency for the registers must be the same as the sampling frequency that is twice the largest frequency of the input signal.

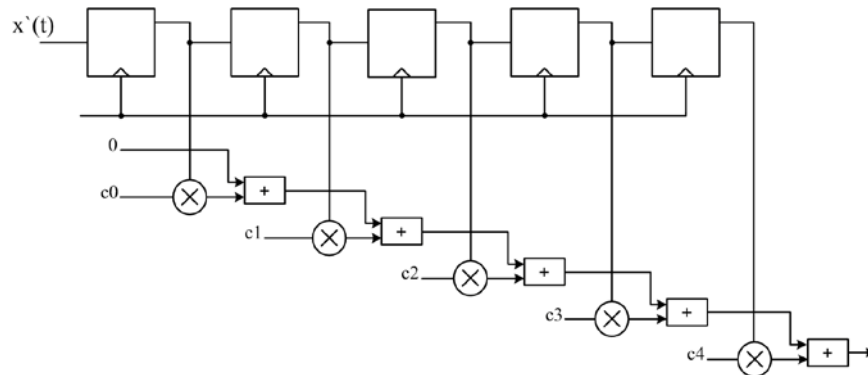


Figure 8.8 RTL FIR Filter

The hardware shown in Figure 8.8 is an iterative hardware and can be described using Verilog **generate** statements. This hardware can easily be implemented using registers, adders and multipliers. Of all these parts, the multipliers are the most complex and take more hardware than the other components.

8.2.2.2 Processor Core FIR Design. The algorithm presented in the block diagram of Figure 8.6 can be implemented with a processor running the iterative add and multiply procedure. A block diagram of this hardware implementation is shown in Figure 8.9.

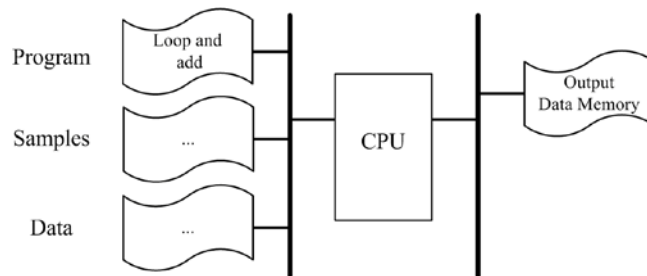


Figure 8.9 Processor Core for FIR Implementation

The data memory of the implementation of Figure 8.9 has a section for storing coefficients and another section for storing sampled data.

We assume that sampled data have been sampled and collected using a valid sampling rate for the input signal.

The program memory of the processor has a program that reads a new sample data performs shifting and five multiply and add operations, one for each coefficient of the filter. With each data read, the program outputs a new data for the circuit response.